

HouseWise

Smart Household & Grocery Management App

Product Requirements Document • MVP v1.0

Document Version	1.0 — MVP Scope
Status	Ready for Engineering
Date	May 2026
Team	Mira (Frontend) Nour (AI Agents) Raghad (QA & Memory) Belal (Backend)
Location	Dubai, UAE
Platform	Mobile App (React Native + Expo + FastAPI)
Target Timeline	2 Weeks to MVP

1. Problem Statement

Managing a multi-person household — for example, a family of three or more adults with a live-in domestic helper — creates a fragmented and inefficient purchasing workflow. Today, item requests are communicated verbally or through informal messages, leading to duplicate purchases, missed items, and the head of household spending significant time and mental energy aggregating and acting on scattered requests.

Existing grocery apps in the UAE offer no cross-store price comparison, no per-unit (price/gram or price/ml) optimization, and no intelligent health-preference filtering. There is currently no solution that aggregates multiple stores and provides optimized purchasing decisions for families in Dubai.

The domestic helper — who manages the kitchen — has no structured tool to flag what is running low, forcing her to rely on memory or informal communication.

2. Solution

HouseWise is a private, invitation-only family mobile application that enables all household members to collaboratively manage a shared shopping list. It features role-specific interfaces, AI-powered price comparison across the household's selected major supermarkets, per-unit cost optimization, and customizable health filters — all culminating in a weekly aggregated report delivered to the head of household via push notification and email.

The system uses an AI browser agent (Playwright-based) that navigates the household's selected supermarkets weekly to fetch live prices, then recommends the optimal purchase strategy — which items to buy from which store — taking into account both price per unit and the family's health preferences.

3. User Personas

3.1 Head of Household — Buyer

- Role: Primary decision-maker and purchaser
- Goal: Review AI price recommendations, select preferred products per item, and receive a single store-grouped shopping list weekly with zero duplication
- Pain point: Spends mental energy aggregating requests; buys duplicates; cannot compare prices across stores
- Interface: Full app access — reviews, edits, approves, and marks lists as purchased

3.2 Family Members

- Role: Household members with personal and shared needs
- Goal: Easily add items during the week without coordinating with others
- Pain point: Requests get forgotten or duplicated
- Interface: Full app access — add items, view list, approve weekly report

3.3 Helper (Cook) — Domestic Staff

- Role: Kitchen manager responsible for cooking and flagging low-stock items
- Goal: Quickly log what is running low or missing with minimal text input
- Pain point: Not tech-savvy; needs a frictionless interface
- Interface: Restricted Helper Mode — two actions only: type item, scan barcode, scan barcode

4. User Stories

4.1 Account & Profiles

1. As a head of household, I want to create one shared family account so that all members operate under the same household context.
2. As a head of household, I want to specify an email address on the account so that the weekly report is delivered to me automatically.
3. As a head of household, I want to select my preferred stores from a list of supported supermarkets so that the price comparison agent only searches stores I actually shop at.
4. As a head of household, I want to suggest a new store so that the team can add it to the supported list in a future release.
5. As a head of household, I want to enter my home address so that the app can display supported stores near me.
6. As any household member, I want to create a named profile with a display name and avatar so that item requests show who made them.
7. As a head of household, I want to assign the Helper role to the domestic staff profile so that she sees a simplified interface.
8. As a head of household, I want to configure health optimization preferences for the household so that AI recommendations reflect our dietary goals.

4.2 Item Requesting

9. As any household member, I want to type an item name to add it to the shared shopping list so that requests are captured digitally.
10. As any household member, I want to scan a product barcode so that the exact product (brand, size) is identified and added with a confirmation screen.
11. As any user, I want to see a confirmation screen after scanning a barcode showing the identified product before it is added so that I can correct misidentifications.

12. As any user, I want to add a quantity and unit to each item request so that the correct amount is purchased.
13. As any user, I want to tag an item as urgent so that the buyer is aware it is needed immediately.
14. As any user, I want to see who requested each item on the aggregated list so that context is preserved.

4.3 Low-Stock Tracking

15. As any household member, I want to flag an item as running low so that it is automatically added to the shared shopping list for the next purchase cycle.

4.4 Weekly Aggregation & AI Price Comparison

16. As the system, I want to automatically trigger a price comparison agent every Thursday night so that Friday's report contains fresh prices.
17. As the system, I want to send the weekly aggregated shopping list to the head of household every Friday morning via push notification and email so that they can review and approve it before purchasing.
18. As the AI agent, I want to navigate each of the household's selected supermarket websites using a headless browser and search for each item so that real prices are fetched without relying on static APIs.
19. As the AI agent, I want to extract product name, package size, and price for each result so that per-unit cost can be calculated.
20. As the AI agent, I want to calculate price per gram or price per ml for each product so that smaller and larger packages are compared fairly.
21. As the system, I want to group the AI-recommended purchase options by store so that the buyer can review and confirm which products to buy from which store.
22. As the buyer, I want to see the estimated total cost per store in the weekly report so that I can budget my shopping trip.
23. As the buyer, I want to see all AI-recommended product options for each item (with price, size, store, and health badge) so that I can make an informed final selection.
24. As the buyer, I want to confirm or swap the AI-selected product for each item before the final purchase list is generated so that I retain full control over what gets bought.

4.5 Health Optimization

25. As a household, I want to configure health preferences (high protein, low carbs, low calories, low sugar, prefer brown over white bread) so that AI recommendations align with our nutritional goals.
26. As the AI agent, I want to label the healthier product option with a 'Healthier Choice' badge so that the buyer can identify it at a glance.
27. As the AI agent, I want to show a cheaper but less healthy alternative with a warning label so that the buyer retains final purchasing control.

4.6 Approval & Purchase Flow

28. As any household member, I want to review and approve the weekly list inside the app so that all members have visibility before it is sent to the buyer for purchase.
29. As the system, I want to send a push notification to all household members when the weekly list is ready for review so that no one misses the approval window.
30. As the system, I want to send a follow-up push notification reminder to household members who have not yet reviewed the list 24 hours before the approval deadline so that the list is not sent incomplete.
31. As the buyer, I want to edit items on the weekly list before approval (remove, change quantity, swap store) so that I have final control.
32. As the buyer, I want to receive a push notification when all household members have reviewed the list so that I know it is ready for my final confirmation and purchase.
33. As the buyer, I want to mark the entire weekly list as purchased so that items are archived to a purchase history log and removed from the active shopping list.
34. As the buyer, I want to mark individual items as purchased or skipped so that partial purchases are tracked accurately and archived to purchase history.
35. As any household member, I want to view purchase history so that I can see what was bought in previous cycles, from which store, and at what price.

5. Functional Requirements

5.1 Authentication & Account Management

- Single shared account per household with one set of credentials
- Named profiles: display name + avatar (emoji or image upload), no separate login per profile — profile is selected on app open or switchable from nav
- Four role types: Admin (Head of Household), Family Member, Helper (Domestic Staff)
- Account-level settings: email for report delivery, weekly aggregation day (default: Friday), health preference configuration, preferred stores (selected from supported list), home address (used to surface nearby supported stores)

5.2 Item Request Module

- Two input methods (text and barcode scan) are available to every household member; Helper Mode surfaces them as the primary interface while family members and admin access them from the item request screen
- Barcode scanner: uses device camera via React Native Camera or Expo Barcode Scanner; calls Open Food Facts or equivalent product database; shows confirmation card with product name, brand, image, size before adding
- Each item request stores: item name, quantity, unit, category, urgency flag, requesting profile name, timestamp

5.3 Low-Stock Tracking

- Low-stock list is a simple list of items flagged as running low by any household member; flagged items are automatically added to the active shopping list
- Auto-clear: when the weekly list is marked purchased, flagged low-stock items are removed from the low-stock list
- Manual flag: any household member can mark an item as running low from the item request screen; Helper sees a dedicated 'Flag Low Stock' tile in Helper Mode
- On item add: system checks if item is already on the active shopping list and shows a soft warning to avoid duplicates

5.4 AI Price Comparison Agent

- Technology: Playwright headless browser agent, scheduled via cron (Thursday 10pm Dubai time)
- Stores covered: user-selected from a supported list; MVP supported stores are Carrefour UAE, Lulu Hypermarket, Union Coop, and Spinneys; agent only queries stores the household has selected; users can suggest additional stores for future releases
- Agent behavior: for each item on the weekly list, navigate to each store website, search for the item, extract top 3 results (name, price, size/weight), calculate price per unit (per gram / per ml / per unit)
- Caching: store extracted prices in the database with a TTL of 7 days; stale prices are flagged in the report
- Fallback: if an item cannot be found on a store, mark as Not Available for that store
- Output: for each item, return up to 3 ranked product options across the selected stores — each showing store, product name, price, size, price-per-unit, and health badge; the buyer reviews these and confirms or swaps before the final purchase list is locked

5.5 Health Preference Engine

- Household-level configuration stored as account settings: toggle switches per preference, applied uniformly across all items and all members
- Preferences: Prefer High Protein, Prefer Low Calories, Prefer Low Carbohydrates, Prefer Low Sugar, Prefer Brown/Whole Grain Bread
- Recommendation logic: top result is the healthiest option meeting preferences; second result is the cheapest option if it differs — shown with a warning badge; all options are presented to the buyer for final selection, not auto-applied

5.6 Weekly Report & Notification

- Generated: every Friday morning (or Monday if configured), after Thursday night price fetch

- Report content: for each requested item, AI-recommended product options grouped by store with per-store estimated totals, health badges, urgency flags, and requesting profile names — buyer reviews and confirms selections before the list is finalised
- Delivery: push notification (via Expo Push Notifications) to all household members when weekly list is ready for review; follow-up push reminder to pending members 24 hours before deadline; email via SendGrid to the buyer containing the weekly report summary and a deep link to the approval screen
- Deep link: opens app directly to the Weekly Approval screen
- In-app approval: any household member (family or buyer) can review and approve; push notification sent to all members when list is ready; follow-up reminder sent to members who have not reviewed 24 hours before deadline; once all members approve, buyer is notified to confirm selections and proceed to purchase
- Post-purchase: buyer taps Mark as Purchased per item or bulk-purchase all; purchased items are archived to purchase history and removed from the active shopping list; skipped items remain on the list for the next cycle

5.7 Helper Mode

- Activated when Helper profile is selected
- Interface: two large tiles — 'Type Item', 'Scan Barcode' — same input methods available to all members, presented in a simplified full-screen layout optimised for low-tech users
- No access to: weekly report, account settings, health preferences, budget data, approval actions
- Low-stock update: Helper sees a third tile — 'Flag Low Stock' — to quickly mark items as running low and add them to the shopping list

6. Non-Functional Requirements

Category	Requirement	Target
Performance	Page load time	< 2 seconds on 4G
Performance	Barcode scan to confirmation	< 3 seconds
Performance	AI price fetch per item	< 10 seconds per store
Reliability	Weekly agent uptime	99% — retry up to 3x on failure
Security	Data storage	All data encrypted at rest; HTTPS only
Usability	Helper Mode task completion	Helper can add item in < 30 seconds
Scalability	Initial user load	5 concurrent users (family scale)

Compatibility	Platform support	iOS 15+ and Android 10+
Localization	Language	English only for MVP

7. High-Level System Architecture

7.1 Technology Stack

Layer	Technology	Rationale
Frontend	React Native (TypeScript) + Expo	Team expertise; native mobile app enables camera, barcode scanning, and push notifications without browser limitations
Backend	FastAPI (Python)	Team expertise; async support; ideal for AI agent orchestration
Database	PostgreSQL	Relational data for users, items, low-stock flags, price cache
Cache	Redis	Price cache TTL, session management
AI Agent	Playwright + Python	Headless browser; handles JS-heavy supermarket sites
LLM Layer	Anthropic API (Claude — claude-sonnet-4)	Product identification, health scoring, report generation
Scheduler	APScheduler / Celery Beat	Thursday night price fetch cron job
Notifications	Expo Push + SendGrid	Push notifications for in-app alerts; email for weekly report delivery
Hosting	Railway / Render (MVP)	Fast deployment; no DevOps overhead for 2-week sprint
Product DB	Open Food Facts API	Barcode lookup for product identification

7.2 Core Modules

- auth-module: account creation, profile management, role assignment
- item-request-module: text/barcode item capture, duplicate list check
- low-stock-module: low-stock flag management, auto-add to shopping list, auto-clear on purchase
- price-agent-module: Playwright browser agent, per-store price extraction, price-per-unit calculation
- health-engine-module: preference configuration, product scoring, badge assignment
- aggregation-module: weekly list compilation, store grouping, report generation

- notification-module: push notification delivery, reminder scheduling, email report delivery via SendGrid
 - approval-module: in-app review, edit, approve, mark-purchased flow, purchase history archiving
-

8. Data Flow

8.1 Item Request Flow

36. User selects input method (type / scan)
37. Item is checked against active shopping list — soft duplicate warning shown if item already present
38. Item saved to item_requests table with: item_name, quantity, unit, category, urgency, requested_by, household_id, status=pending, created_at
39. Item appears in shared Active Requests list visible to all family members

8.2 Weekly Aggregation Flow (Thursday Night)

40. Cron job triggers price-agent-module
41. Agent fetches all pending item_requests for the household
42. For each item: agent navigates the household's selected stores — searches item — extracts top results — calculates price_per_unit — stores in price_cache table
43. Health engine scores each result against household health preferences — assigns badges
44. Aggregation module compiles AI-recommended options per item — calculates per-store estimated totals — presents to buyer for review and confirmation before generating final purchase list
45. Weekly report record created in weekly_reports table
46. Notification module sends push notification to all household members with deep link to review screen; schedules follow-up reminder for members who have not approved

8.3 Approval & Purchase Flow

47. Any household member opens deep link — lands on Weekly Approval screen
 48. Member reviews list — can edit quantities or flag items
 49. Member taps Approve — approval recorded — once all members have approved, buyer receives push notification to confirm and proceed to purchase
 50. Buyer completes shopping — marks items as purchased individually or in bulk
 51. System: purchased items archived to purchase_history table — removed from active shopping list — item_requests status updated to purchased — weekly report status set to completed — low-stock flags for purchased items cleared
-

9. API Design (FastAPI Endpoints)

Method + Path	Description	Auth
POST /auth/account	Create household account	Public
POST /auth/profile	Create named profile	Household token
GET /profiles	List household profiles	Household token
POST /items	Add item request	Profile token
GET /items	Get all active item requests	Profile token
PATCH /items/{id}	Update item (qty, urgency, status)	Profile token
DELETE /items/{id}	Remove item request	Profile token
GET /low-stock	Get low-stock items	Profile token
PATCH /low-stock/{item_id}	Flag or clear low-stock item	Helper / Admin
POST /agent/run	Trigger price fetch manually	Admin only
GET /reports/latest	Get latest weekly report	Profile token
POST /reports/{id}/approve	Approve weekly report	Profile token
PATCH /reports/{id}/purchase	Mark items as purchased	Admin / Family
GET /health-prefs	Get household health preferences	Profile token
PUT /health-prefs	Update health preferences	Admin only

10. Implementation Decisions

10.1 Architecture Decisions

- **Decision:** Single shared account model

All household members share one account and one data context. Profiles are display-level only — no separate authentication per user. Profile is selected or switched in the app without a password. This eliminates onboarding friction for a 2-week MVP.

- **Decision:** React Native mobile app over responsive web

Native mobile app (React Native + Expo) is the MVP platform. Enables native camera access for barcode scanning, push notifications, and a better UX for the Helper role. Deployable to iOS and Android via Expo. A web admin dashboard is a post-MVP consideration.

- **Decision:** Playwright headless browser agent over scraping or APIs

UAE supermarket chains (Carrefour, Lulu, Union Coop, Spinneys) do not expose public APIs. Playwright handles JavaScript-rendered pages reliably. Agent runs on Thursday night to minimize detection risk and ensure fresh Friday data. The supported store list is maintained in config — new stores can be added without changing core agent logic.

- **Decision:** Price-per-unit as primary comparison metric

Raw price comparison is misleading (500g vs 1kg). All price comparisons normalize to price-per-gram for solids and price-per-ml for liquids. Unit detection is extracted from product title using regex + LLM fallback.

- **Decision:** Anthropic API (Claude) for product scoring and report generation

The LLM is used for: (1) barcode confirmation product description, (2) health preference scoring when nutritional data is unavailable from Open Food Facts, (3) natural-language weekly report generation.

10.2 Schema Sketch

- households: id, name, email, health_prefs (JSON), aggregation_day, preferred_stores (JSON), home_address, created_at
- profiles: id, household_id, display_name, avatar_url, role (admin | family | helper), created_at
- item_requests: id, household_id, requested_by_profile_id, item_name, quantity, unit, category, urgency (bool), status (pending | approved | purchased | skipped), barcode, created_at
- low_stock_flags: id, household_id, item_name, flagged_by_profile_id, flagged_at, cleared_at (nullable)
- price_cache: id, household_id, item_name, store, product_name, price_aed, size, unit_type, price_per_unit, health_score, fetched_at, ttl_expires_at
- weekly_reports: id, household_id, generated_at, status (pending | approved | completed), report_data (JSON), approved_by_profile_id, approved_at
- purchase_history: id, household_id, item_request_id, item_name, quantity, unit, store, price_paid, purchased_by_profile_id, purchased_at, weekly_report_id

11. Testing Decisions (Raghad — QA Lead)

What makes a good test

Tests cover external behavior and API contracts, not internal implementation. A test should verify that given an input, the system produces the correct output — not how it does so internally. Tests should be repeatable, fast, and not dependent on live supermarket websites.

Modules to test

- price-agent-module: mock Playwright responses with recorded HTML fixtures from each store; verify price extraction and per-unit calculation correctness
- health-engine-module: unit test scoring logic with known product nutrition data; verify badge assignment against configured preferences
- aggregation-module: given a set of price_cache records, verify correct store grouping and total calculation

- item-request-module: test barcode lookup, duplicate list detection
- approval-module: test state transitions — pending → approved → completed — and auto-clear of low-stock flags on purchase
- notification-module: mock SendGrid + Expo Push; verify message content and deep link correctness

QA Scope for MVP (2 weeks)

- Manual testing: full happy-path flow on iOS (iPhone) and Android devices
- Automated: API integration tests for all FastAPI endpoints using pytest
- Agent testing: dry-run mode for Playwright agent using cached HTML fixtures — no live site calls in CI

12. MVP Scope vs Post-MVP

Feature	MVP	Post-MVP
Shared account + named profiles	YES	
Text / barcode item add	YES	
Low-stock tracking	YES	
AI price comparison (user-selected stores)	YES	
Price-per-unit normalization	YES	
Health preference engine (soft recommendations)	YES	
Weekly aggregated report	YES	
Push + email notification	YES	
In-app approval + mark purchased	YES	
Helper Mode (restricted interface)	YES	
Amazon Now integration		POST-MVP
Recipe / Cookbook feature		POST-MVP
Web admin dashboard		POST-MVP
Budget tracking and spend history		POST-MVP
Meal planning integration		POST-MVP
Multi-household support		POST-MVP

Push notifications (native)		POST-MVP
-----------------------------	--	----------

13. Out of Scope (MVP)

- Recipe book and cookbook feature — full post-MVP
- Amazon Now / delivery app integrations — post-MVP
- Web application — mobile app only for MVP
- Multi-household or multi-family accounts
- Payment processing or direct order placement — report-and-buy model only
- Real-time price updates (prices fetched weekly only)
- Social or sharing features

14. Risks, Assumptions & Constraints

Risks

- Supermarket website scraping: Carrefour/Lulu may block Playwright agents. Mitigation: use stealth mode plugins, randomize user-agent, add rate limiting, cache aggressively. If blocked, fall back to cached prices with a staleness warning.
- 2-week timeline is extremely tight. Mitigation: frontend and backend must work from agreed API contracts from Day 1; Noor's AI agent is the highest-risk component — start Day 1.
- Open Food Facts may not have UAE-specific barcode data. Mitigation: integrate a secondary database (e.g., Buycott, UPC Item DB) or allow manual product confirmation.
- Nutritional data for UAE products is sparse. Mitigation: LLM-based health scoring as fallback when structured nutritional data is unavailable.

Assumptions

- The household has a stable internet connection in Dubai
- The helper owns or has access to a smartphone capable of running the mobile app
- Supermarket websites are publicly accessible without login for product search
- The family's health preferences are stable and change infrequently

Constraints

- 2-week MVP delivery window — scope must stay strictly within MVP list
- No native app submission — App Store / Play Store review takes 1-2 weeks
- No dedicated DevOps engineer — use managed hosting (Railway/Render)

- English-only interface for MVP

15. Team Responsibilities

Team Member	Role	Primary Ownership
Mira	Frontend Engineer	React Native UI, Helper Mode, approval flow, responsive design
Noor	AI Agent Engineer	Playwright price agent, Anthropic API integration, health scoring engine, report generation
Rapid	QA & Memory	API integration tests, agent dry-run fixtures, low-stock state management, data schema
Product Owner	Product Owner	Requirements, acceptance criteria, UAT, stakeholder sign-off

Further Notes

- The weekly aggregation day defaults to Friday (day before UAE weekend). Monday is the alternative configurable option.
- The PRD is a living document. Post-MVP features (Amazon Now, recipe book, native app) should be revisited after the first 4 weeks of usage data.